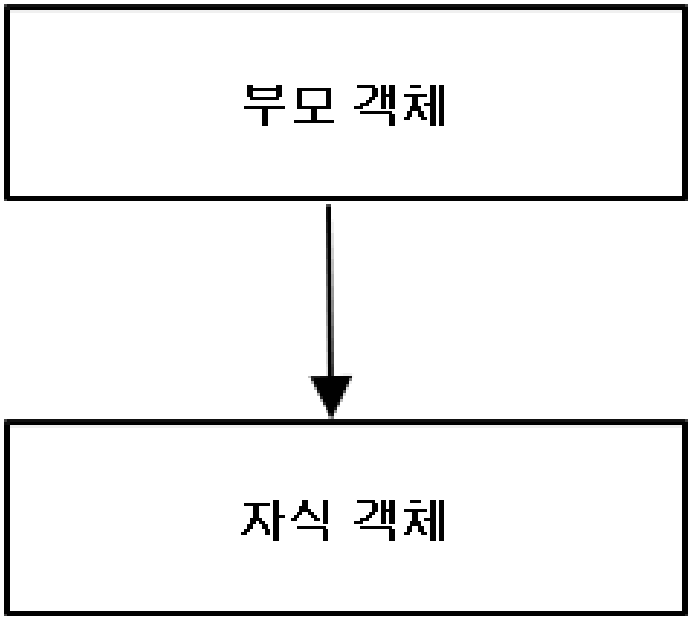
안녕하세요.
Javascript 강좌중인 Falcon 입니다.

아마 이 부분은 정리 글이긴 하지만 가장 이해하기 힘든 부분이 될 거 같습니다. 특히 Javascript가 처음인 분들은 더 이해가 힘든 부분일 수 있을 거 같다는 생각이 들지만 일단 설명을 해보도록 하겠습니다.



상속이라는 것은 쉽게 얘기하면 부모가 가진 부분들을 그대로 물려받아서 자식인 객체가 사용하는 기능 입니다.
어디에 이런 기능을 쓰는지는 여러분들이 계속해서 공부를 하다보면 알게 될테니 지금은 이정도만 알면 됩니다.

```
8         <script type="text/javascript">
9           function person(nick) {
10            this.nick = nick;
11            this.showData = function() {
12              return 'My nick is ' + this.nick;
13            }
14          }
15
16          function programmer(nick) {
17            this.nick = nick;
18          }
19          programmer.prototype = new person();
20
21          var p1 = new programmer('Falcon');
22          console.log(p1.showData());
23        </script>
```

오늘은 상속에 대해서 배워볼텐데 먼저 person 함수객체가 있습니다. 이 객체는 부모 객체가 될 객체 입니다.

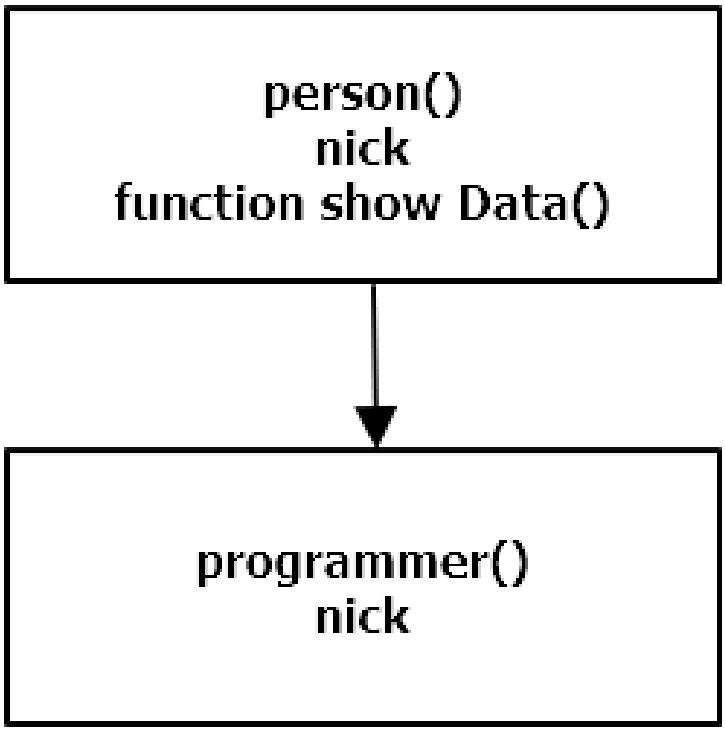
```
9           function person(nick) {
10            this.nick = nick;
11            this.showData = function() {
12              return 'My nick is ' + this.nick;
13            }
14          }
```

이 객체는 이전강좌에서 배운 new와 생성자 타입의 객체 입니다. 상속이라는 것은 부모가 가진 객체를 그대로 물려받는다고 위에서 말씀드렸습니다. 그럼 새로운 객체가 있어야 한다는 것을 의미 합니다.

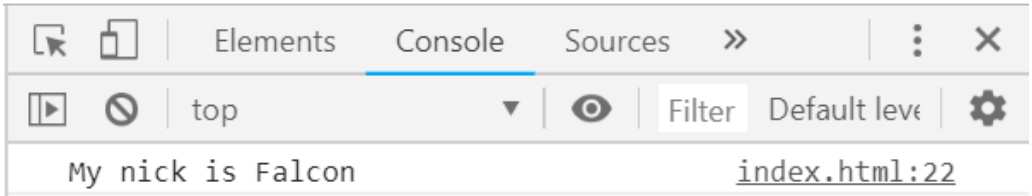
```
16          function programmer(nick) {
17            this.nick = nick;
18          }
19          programmer.prototype = new person();
```

밑에 보시면 programmer이라는 객체를 하나 더 만들었습니다. 기본 적으로 생성자로 오는 것은 같아야합니다. 이 부분이 존재하지 않으면 nick 선언이 불가능하기 때문 입니다. 19행을 보시면 prototype라는 객체가 보입니다. 이 객체는 여러분들이 상속할 객체를 의미합니다. new person을 한번 봅시다.

new person은 새로운 공간안에 이렇게 객체를 생성했고 이를 programmer이 상속 받고 있는 것 입니다. 쉽게 말해서 person은 부모객체이고 programmer은 자식객체라는 것입니다. 그걸 선언하는 객체가 prototype라는 것 입니다. 즉 19행 부분이 선언된 이후로는 programmer은 person의 자식 객체가 되는 겁니다.



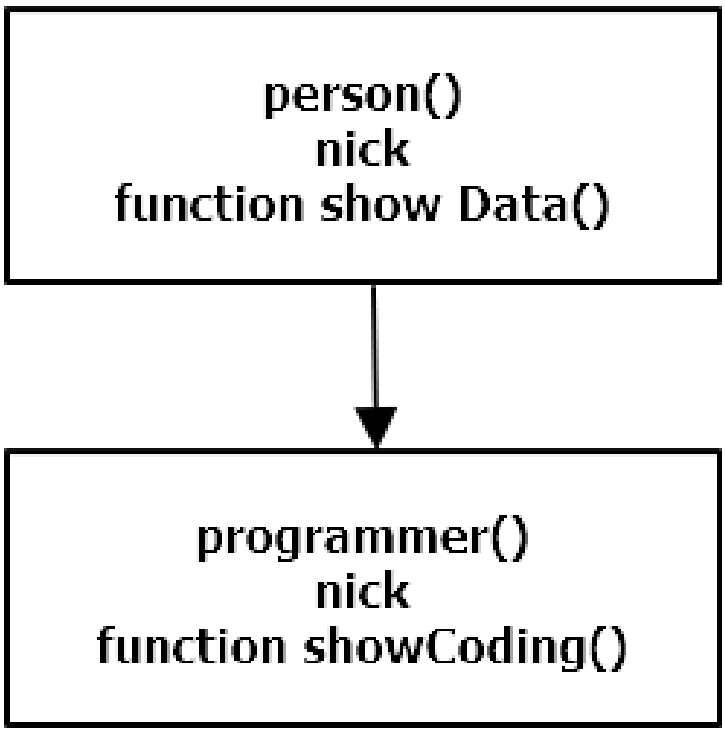
여기서 보시면 programmer이라는 객체에는 nick만 존재합니다. 그런데 person의 기능적인 showData를 programmer 객체가 사용할 수 있습니다. 그 출력방법이 22행에서 나타내고 있습니다.



위의 코드를 실행해보면 정상적으로 실행되는 것을 볼 수 있습니다. prototype이라는 객체를 본다면 여러분들은 이제부터 어딘가에 이 객체는 자식객체이며 부모 객체가 존재하는구나라는 것을 알 수 있어야 합니다.

```
8         <script type="text/javascript">
9           function person(nick) {
10            this.nick = nick;
11            this.showData = function() {
12              return 'My nick is ' + this.nick;
13            }
14          }
15
16          function programmer(nick) {
17            this.nick = nick;
18            this.showCoding = function() {
19              return 'coding';
20            }
21          }
22          programmer.prototype = new person();
23
24          var p1 = new programmer('Falcon');
25          console.log(p1.showData());
26          console.log(p1.showCoding());
27        </script>
```

윗 부분과 다든점은 programmer에 함수가 하나 추가된 것 외에는 없습니다. showCoding이라는 함수가 추가되었는데 제가 여러분들에게 알려드리고 싶은 것은 person 객체를 부모 객체로 두면서 programmer은 자신만의 기능을 가질 수 있다는 것을 보여드리려는 겁니다. 즉, 위의 코딩은 그림으로 그리면 이렇게 됩니다.



그림으로 그려보면 이렇게 programmer 객체 안에만 showCoding이라는 기능이 추가 될 수 있습니다.

```
28          var p2 = new person('Spider');
29          console.log(p2.showData());
30          console.log(p2.showCoding());
```

위는 개인적으로 제가 추가해서 보여드리는 것입니다. 여러분들도 추가를 해보셔서 확인해보시길 바랍니다.
이 상속의 좋은 점중의 하나는 person도 살아있다는 것입니다. person은 person만의 기능을 사용할 수 있습니다.
바로 위처럼 말이죠. 다만 30행에 보이는 showCoding은 사용할 수 없습니다. person 자체가 부모이긴 하지만
prototype으로 programmer객체가 자식 객체라할지라도 부모 객체는 자식객체의 기능을 가져다 쓸 수 없습니다.

```
Elements   Console   Sources   »   1
top   Filter   Default leve
My nick is Falcon                          index.html:25
coding                                     index.html:26
My nick is Spider                          index.html:29
▸Uncaught TypeError: p2.showCoding is      index.html:30
not a function
    at index.html:30
```

이를 실행해보면 30행 부분에 오류가 있다고 알려줍니다. 나머지는 정상적으로 출력되는 것을 볼 수 있습니다.

1. prototype라는 객체를 사용하면 상속 기능을 사용할 수 있다.

2. 부모 객체를 상속했을 경우 자식 객체는 부모 객체의 기능을 사용할 수 있다.
3. 부모 객체를 선언할 경우에는 자식 객체의 기능을 사용할 수 없다.

위의 내용을 정리하면 이정도가 될 거 같습니다. 여러분들도 직접 코딩해서 실행을 해보시길 바랍니다.

이것으로 [Javascript 강좌] 제 24강 상속에 대해서 알아보자 편을
마치도록 하겠습니다.